SmartBus Astana

Project Integration, Testing and Improvement · Endterm

User goal: Help a passenger choose a less crowded departure: select a route, stop and time, compare predictions, then save the preferred option.

Scope: A working HTML/CSS/JavaScript prototype using simulated demand rules. No live CTS data, physical sensors or trained machine-learning model.

Team: Amangeldi Alikhan · Kerimbayev Akarys · Kartbay Alim

Group: [Group number — to be provided]

Same project as Assignment 4. Technical baseline: 1c2b3a7. The team supplied amanalikhan9020.github.io/smart-bus-v2/ as its A4 website; the exact submitted commit is not confirmed.

Functional progress & connected scenario

New functionality: compare three departure times and save the selected prediction.

Before: one prediction. After: the selected time, +30 and +60 minutes, with occupancy, passenger counts and a recommendation. Equal predictions are reported honestly.

Guest / login → route + stop + time → prediction → compare → select → save → Saved Trips → reload → retrieve the same choice.

Data flow: Inputs → shared calculation model → selected trip object → localStorage keyed by user ID → trip list and saved-data analytics.

Crossing midnight is labelled ‘Next day’. These are simulated demand comparisons, not real bus departure schedules.

The existing simulated route-demand chart remains a separate analysis. Not every page needs to be part of the same scenario.

Previous functionality: demonstrate live

19 automated browser tests passed (33.690 s, exit 0). Automated tests are not human usability tests.

Area	Working demonstration
Data storage	Save and retrieve after reload; filter and delete saved trips.
Validation	Reject empty/whitespace-only required input, name the field, prevent saving, then succeed after correction. Also check email format and duplicates.
Smart function	Input → occupancy and passengers. Bus 10, first stop: 18:00 → 56%; 05:00 → 10%. The same model powers comparison.
Data analysis	Average + maximum + a chart from saved predictions. Adding/deleting trips updates the results; empty data has a clear state.
Basic security	HTML input stays inert; wrong passwords are blocked. Salted PBKDF2, account separation and legacy migration/recovery. Local protection only.

Illustrative usability results

Two simulated records for the same task. No real participants were tested.

T1: Continue as guest. Choose Bus 10, the first stop and 18:00. Compare three options, save the least crowded, find it after reload, then report the saved average and maximum occupancy.

User	Task	Completion result	Difficulties / feedback
Sample 1	T1	Completed independently	Initially missed the departure-time field. Suggested a more visible time label.
Sample 2	T1	Completed with help	Needed an explanation of occupancy percentages. Suggested showing the level thresholds.

These example outcomes describe possible usability problems; they are not observations collected from people.

To satisfy the assessment, give T1 to two people outside the team and replace these samples with their actual results.

Two implemented usability improvements

Illustrative feedback → implemented audit improvement → technical verification.

Sample problem	Before / after and technical verification
1. Time selection was hard to find	Before: a generic ‘Time’ label. After: ‘Departure time (required)’, an accented field and a clear missing-time error. Checked: visible on a 390px screen; changing 18:00 to 19:00 updates the prediction from 56% to 49%.
2. Occupancy meaning was unclear	Before: percentage and level without numeric thresholds. After: passenger/capacity definition; low <45%, medium 45–74%, high ≥75%. Checked: explanation is visible; 05:00 gives 10%/Low, 18:00 gives 56%/Medium for Bus 10, first stop.

These changes are real and were found through code review. The sample feedback is simulated; no improvement in human task completion or understanding has been measured.

The 19 automated regression checks passed. Real feedback and a repeated affected task are still required to complete the user-testing criterion.

Limitations, next steps & team contributions

Live presentation and demonstration: 8–10 minutes, followed by questions and functional checks.

Simulated occupancy, no live CTS or guaranteed arrival times. Local data has no device sync or server authentication. The browser owner can edit storage; PBKDF2 does not encrypt localStorage.

Next steps: Confirm the A3–A4 baseline; run two real user tests; implement two feedback-based improvements and repeat the tasks. Before Final: evaluate the model, consider verified data sources and server-side storage/access checks.

Contributions: Reported A4 work: Alikhan 33.3% — core logic/predictions; Akarys 33.3% — UI/charts; Alim 33.4% — validation/security/tests. Each member’s technical contribution AFTER A4: [confirm].

Every member must understand the complete scenario, explain relevant code/formulas/tests and answer individually. Slides or speaking alone are not enough; AI is allowed when the team understands the implementation.

Moodle: repository link OR source archive + final PDF (4–6 slides) + all names and group number. A separate report is not required. This draft still needs the missing evidence.